

Inventory System

Design Document • v1.0

NestJS • Prisma • Next.js PWA

1. Project Overview

A self-hosted Progressive Web Application for managing household inventory across multiple locations. The system supports hierarchical lists (e.g. Basement → Pantry → Spices), per-user access control, and barcode/QR code scanning for item check-in and check-out. Designed for a small household with two user roles: Admin and limited-access Family members.

1.1 Goals

- Maintain a real-time inventory of items across the home.
- Support hierarchical list structure with unlimited depth.
- Allow role-based access: Admin sees everything; other users see only shared lists.
- Enable QR/barcode scanning via phone camera to quickly update stock.
- Work offline via PWA service worker; sync when back online.
- Print QR code labels from within the application.

1.2 Non-Goals

- Cloud hosting — system is designed to run locally (home server, NAS, or Raspberry Pi).
- E-commerce or purchasing integrations.
- Complex reporting or analytics.

2. Technology Stack

Layer	Technology	Purpose
Backend API	NestJS	RESTful API with guards, pipes, and modules
ORM / DB	Prisma + PostgreSQL	Type-safe database access and migrations
Frontend	Next.js (App Router)	Server and client components, API routes
PWA	next-pwa / Workbox	Service worker, offline cache, installable
Auth	NextAuth.js (credentials)	Session management and JWT
QR / Barcode	html5-qrcode	Camera-based scanning in browser
QR Generation	qrcode npm	Generate QR codes server-side
Styling	Tailwind CSS	Utility-first styling
State	React Query (TanStack)	Server state, cache, optimistic updates
Labels	Browser Print / PDF	Print QR labels directly from the PWA

3. Data Model

3.1 Entity Relationship Summary

Below are the primary Prisma models. All IDs are cuid() strings. Timestamps are managed automatically.

User

Field	Type	Notes
id	String (cuid)	Primary key
name	String	Display name
email	String (unique)	Used for login
passwordHash	String	bcrypt hashed
role	Enum	ADMIN EDITOR VIEWER
createdAt	DateTime	Auto-managed
lists	ListAccess[]	Join table — lists this user can access

List

Field	Type	Notes
id	String (cuid)	Primary key
name	String	e.g. 'Pantry', 'Tools'
description	String?	Optional description
parentId	String?	Self-relation — null means top-level
parent	List?	Parent list relation
children	List[]	Sub-lists
items	Item[]	Items in this list
ownerId	String	User who created the list
accesses	ListAccess[]	Users who can see this list
createdAt	DateTime	Auto-managed

Item

Field	Type	Notes
id	String (cuid)	Primary key

Field	Type	Notes
name	String	Item name
description	String?	Optional description
imageUrl	String?	Stored image path or URL
quantity	Int	Current stock count
unit	String?	e.g. 'kg', 'pcs', 'L'
barcode	String?	Scanned barcode value
qrCode	String?	Generated QR identifier
listId	String	Parent list
createdAt	DateTime	Auto-managed
updatedAt	DateTime	Auto-managed

ListAccess (Join Table)

Field	Type	Notes
id	String (cuid)	Primary key
userId	String	User being granted access
listId	String	List being shared
permission	Enum	READ WRITE
grantedById	String	Admin who granted access
createdAt	DateTime	Auto-managed

Invite

Field	Type	Notes
id	String (cuid)	Primary key
token	String (unique)	Secure random token
email	String?	Target email (optional)
role	Enum	Role assigned on accept
expiresAt	DateTime	Invite TTL (24–72 hrs)
usedAt	DateTime?	Null if not yet used
createdById	String	Admin who created it

Transaction (Audit Log)

Field	Type	Notes
id	String (cuid)	Primary key
itemId	String	Item affected
userId	String	User who made the change
type	Enum	IN OUT ADJUST DELETE
delta	Int	Quantity change (+/-)
note	String?	Optional scan or manual note
createdAt	DateTime	Auto-managed

4. Application Architecture

4.1 NestJS Backend Modules

Module	Responsibilities
AuthModule	Login, JWT strategy, session validation, invite accept flow
UsersModule	CRUD for users, role management, removal by admin
ListsModule	Create/read/update/delete lists and sub-lists, access control
ItemsModule	CRUD for items, quantity adjustments, QR/barcode association
AccessModule	Manage ListAccess records — share/unshare lists with users
InvitesModule	Generate invite tokens, validate on accept, expire tokens
TransactionsModule	Log IN/OUT/ADJUST events, serve history per item or list
QrModule	Generate QR code strings and PNG buffers for label printing
UploadsModule	Handle item image uploads, store locally or on NAS

4.2 Access Control Flow

Every API request passes through two layers:

- JwtAuthGuard — validates the JWT and attaches the user to the request.
- RolesGuard — checks the user's global role (ADMIN / EDITOR / VIEWER) against the required role on the endpoint decorator.
- ListAccessGuard — for list-scoped endpoints, verifies the user has an active ListAccess record for that listId (or is ADMIN).

Admins bypass ListAccessGuard entirely — they can access all lists, items, and users.

4.3 Next.js Frontend Structure

Route	View	Access
/	Dashboard — all accessible lists	All roles
/lists/[id]	List detail — items + sub-lists	Access required
/lists/[id]/all	Aggregated view — list + all children	Access required

Route	View	Access
/lists/new	Create list form	EDITOR or ADMIN
/items/[id]	Item detail — quantity, history, QR	Access required
/scan	Camera scanner — scan to find/update item	All roles
/admin/users	Manage users, roles, remove	ADMIN only
/admin/invites	Generate & copy invite links	ADMIN only
/invite/[token]	Accept invite, create account	Public (token-gated)
/settings	User profile, preferences	All roles

5. Feature Specifications

5.1 Lists

- A list has a name, optional description, and an optional parentId.
- Top-level lists have no parent (parentId is null).
- Sub-lists inherit visibility: if a user can see 'Basement', they see 'Basement → Pantry' only if separately shared, OR if the admin grants access to the parent with cascade flag.
- The owner (creator) always retains access regardless of ListAccess records.
- Admins can delete any list; editors/viewers can only delete lists they own.
- Deleting a list cascades: all sub-lists and their items are also deleted (with confirmation dialog).

5.2 Items

- Items belong to exactly one list.
- An item can have: name (required), description, image, quantity, unit, barcode, QR code.
- Quantity can be increased (+), decreased (-), or set absolutely via a manual adjust.
- Quantity cannot go below 0. A dedicated 'Remove Item' action sets quantity to 0 and marks the item as archived, or hard-deletes (admin choice).
- Every quantity change writes a Transaction record for audit trail.
- Items can be moved between lists by an ADMIN or the list owner.

5.3 QR & Barcode Scanning

- The /scan page opens the device camera using html5-qrcode.
- Scanning a known QR code (generated by the app) navigates to that item's detail page and shows quick +/- controls.
- Scanning an external barcode (e.g. food product) searches existing items first; if not found, opens the 'Add Item' form pre-filled with the barcode value.
- The app can optionally query the Open Food Facts API to auto-fill name and image for pantry items.
- QR codes are generated server-side as PNG and can be downloaded or printed as labels.
- Each QR code encodes the item ID as a URL: `https://app.local/items/{id}`.

5.4 Aggregated List View

- The '/lists/[id]/all' route recursively fetches all items from a list and its descendants.
- A 'Show All' button on any list card navigates to this view.
- Items are grouped by their immediate parent list name for readability.

- Filtering by name is available in this view.

5.5 User Management (Admin)

- Admin can generate invite links with a selected role (EDITOR or VIEWER).
- Invite links are single-use, expire after 72 hours, and can be revoked early.
- On accepting an invite, the new user sets their name and password.
- Admin can change any user's role at any time.
- Admin can remove a user — their ListAccess records are deleted; their lists are reassigned to admin or deleted (choice at removal time).
- Admin can share any list with any user and set READ or WRITE permission.

5.6 PWA & Offline

- The app is installable on both Android (Chrome) and iOS (Safari 16.4+).
- A Workbox service worker caches the app shell and the user's list/item data.
- Quantity changes made offline are queued in IndexedDB and synced on reconnect.
- The camera scanner requires an active internet connection only for the Open Food Facts lookup; QR scanning works offline.
- A visual indicator shows the offline/online state.

6. API Endpoint Reference

6.1 Auth

Method	Endpoint	Description	Access
POST	/auth/login	Email + password → JWT	Public
POST	/auth/logout	Invalidate session	Authenticated
POST	/auth/invite/accept	Accept invite token, create user	Public (token)

6.2 Users

Method	Endpoint	Description	Access
GET	/users	List all users	ADMIN
PATCH	/users/:id/role	Change role	ADMIN
DELETE	/users/:id	Remove user	ADMIN

6.3 Lists

Method	Endpoint	Description	Access
GET	/lists	All accessible lists (flat)	Authenticated
GET	/lists/:id	Single list + immediate children	Access required
GET	/lists/:id/all	List + all descendants + items	Access required
POST	/lists	Create list (parentId optional)	EDITOR / ADMIN
PATCH	/lists/:id	Update name/description	Owner / ADMIN
DELETE	/lists/:id	Delete list (cascades)	Owner / ADMIN

6.4 Items

Method	Endpoint	Description	Access
GET	/lists/:id/items	Items in a list	Access required
POST	/lists/:id/items	Add item to list	EDITOR / ADMIN
GET	/items/:id	Item detail	Access required
PATCH	/items/:id	Update name/description/image	EDITOR / ADMIN
PATCH	/items/:id/quantity	Adjust quantity (+/-/set)	EDITOR / ADMIN
DELETE	/items/:id	Remove item	Owner / ADMIN
GET	/items/:id/qr	QR code PNG	Access required
GET	/items/scan/:code	Lookup by barcode or QR value	Authenticated

6.5 Access & Invites

Method	Endpoint	Description	Access
POST	/lists/:id/access	Share list with user	ADMIN
DELETE	/lists/:id/access/:userId	Revoke list access	ADMIN
POST	/invites	Generate invite link	ADMIN
DELETE	/invites/:id	Revoke invite	ADMIN

7. UI/UX Design Principles

7.1 Navigation

- Bottom navigation bar on mobile: Home, Scan, Lists, Admin (admin only).
- Breadcrumb trail on list/item detail pages: e.g. Basement → Pantry → Item.
- Each list card shows: name, item count, sub-list count, and a 'Show All' button.

7.2 List Detail Page

- Two tabs: Items | Sub-lists.
- Items tab: card-based grid with item name, image thumbnail, quantity controls (+/-), and a 'Remove' button.
- Sub-lists tab: card grid of child lists with the same 'Show All' shortcut.
- Floating '+' button to add a new item or sub-list.

7.3 Scanner Page

- Full-screen camera viewfinder.
- After a successful scan, a bottom sheet slides up showing the item name, current quantity, and quick +/- buttons.
- 'Unknown code' shows a prompt to add a new item with that code pre-filled.

7.4 Admin Panel

- Users table with role badge, 'Change Role' dropdown, and 'Remove' button.
- Invites section: 'Generate Link' button, list of active invite links with copy/revoke actions.
- List sharing: search for a user, pick a list, set READ or WRITE, confirm.

8. Open Questions & Decisions Needed

#	Status	Question	Decision
1	 Resolved	Hosting environment	MVP on dev laptop; later migrate to Raspberry Pi 5 (8GB) with USB SSD
2	 Resolved	HTTPS requirement	Tailscale + MagicDNS for all environments — see Section 10
3	 Resolved	Server strategy	Docker Compose + Caddy reverse proxy — see Section 11
4	 Pending	Image storage	Local filesystem on the server, or a mounted NAS share. No S3 for self-hosted.
5	 Pending	List access inheritance	When sharing 'Basement', does that auto-share 'Basement → Pantry'? Or must each sub-list be shared separately?
6	 Pending	Offline write conflict	If both users adjust quantity offline, which wins on sync? Last-write-wins vs. delta-merge (sum of deltas).
7	 Pending	Label printer integration	Print from browser (PDF label sheet) or Phomemo/NIIMBOT via Web Bluetooth API?
8	 Pending	Open Food Facts	Use for pantry auto-fill? Should it be opt-in per list or per item?

9. Suggested Build Milestones

Phase	Scope	Outcome
0 — Local Dev	Docker Compose on laptop, Tailscale, mkcert for HTTPS	Full local environment running with HTTPS camera access
1 — Core API	Prisma schema, Auth, Users, Lists, Items endpoints	Working REST API with auth and access control
2 — Frontend Shell	Next.js layout, auth pages, list/item pages, React Query hooks	Usable web app, no PWA yet
3 — PWA + Scanner	Service worker, offline queue, /scan camera page	Installable on phones, camera scanning works
4 — Admin Panel	User management, invite flow, list sharing UI	Full multi-user experience
5 — QR Labels	QR generation, label print page, barcode scan lookup	End-to-end scan workflow
6 — Polish	Offline sync conflict handling, image uploads, Open Food Facts	Production-ready
7 — Pi Migration	Provision Raspberry Pi 5, move Docker stacks, point Tailscale	Permanent always-on home server

10. HTTPS & Tailscale Setup

The PWA camera API (used for QR/barcode scanning) requires HTTPS — it will not work on plain HTTP even on a local network. Tailscale is the recommended solution for all environments: laptop MVP and the eventual Raspberry Pi.

10.1 Why Tailscale

- Creates an encrypted private network between your devices (laptop/Pi, your phone, wife's phone).
- Every device gets a stable 100.x.x.x IP and a human-readable hostname via MagicDNS.
- Tailscale provides free, auto-renewing HTTPS certificates for your devices — no self-signed certs, no browser warnings.
- Works at home (LAN speeds) and away from home (routed via Tailscale relay).
- Free tier supports up to 3 users and 100 devices — more than enough.
- 10-minute setup. No port forwarding, no router config, no dynamic DNS.

10.2 MVP Setup on Laptop

Step 1 — Install Tailscale

Install Tailscale on your laptop and both phones from tailscale.com/download. Sign in with the same account on all devices. They will appear in your Tailscale admin panel automatically.

Step 2 — Enable HTTPS in Tailscale Admin

In the Tailscale admin console (admin.tailscale.com), go to DNS → enable MagicDNS and enable HTTPS certificates. Your laptop will get a hostname like: `mypc.tail1234.ts.net`

Step 3 — Provision the Certificate on Your Laptop

Run the following command once on your laptop:

```
tailscale cert mypc.tail1234.ts.net
```

This generates a `cert.pem` and `key.pem` in your working directory. Point Caddy at these files and it will serve HTTPS automatically.

Step 4 — Access from Phones

Install Tailscale on both phones and sign in. The app is then accessible at <https://mypc.tail1234.ts.net> with a valid certificate — camera access works, PWA install works, no warnings.

10.3 Alternatives Considered

Option	Pros	Cons	Verdict
Tailscale	Zero config HTTPS, works remotely, free	Requires account	✅ Recommended
mkcert	Fully offline, no account	Must install CA cert on every phone manually — fiddly on iOS	Backup option
Cloudflare Tunnel	Public URL, real cert	Traffic routes through Cloudflare servers	Overkill for home use
Self-signed cert	No dependencies	Browser and PWA reject it; camera blocked	❌ Do not use

11. Infrastructure & Deployment

11.1 Overview

All services run in Docker containers managed by Docker Compose. Caddy acts as the single reverse proxy, terminating HTTPS and routing traffic to each service by subdomain. This same setup runs identically on the laptop (MVP) and on the Raspberry Pi (production).

11.2 Service Architecture

Container	Image	Internal Port	Purpose
caddy	caddy:2-alpine	80, 443	Reverse proxy, HTTPS termination
nextjs	custom build	3000	Next.js PWA frontend
nestjs	custom build	4000	NestJS REST API
postgres	postgres:16-alpine	5432	Primary database

11.3 Docker Compose File

```
# docker-compose.yml
version: '3.9'
services:
  caddy:
    image: caddy:2-alpine
    ports: ['80:80', '443:443']
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile
      - ./certs:/certs:ro
      - caddy_data:/data

  nextjs:
    build: ./frontend
    environment:
      - NEXTAUTH_URL=https://inventory.mypc.tail1234.ts.net
      - API_URL=http://nestjs:4000

  nestjs:
    build: ./backend
    environment:
      - DATABASE_URL=postgres://postgres:secret@postgres:5432/inventory
    depends_on: [postgres]

  postgres:
    image: postgres:16-alpine
    volumes: [pg_data:/var/lib/postgresql/data]
    environment:
      - POSTGRES_PASSWORD=secret
      - POSTGRES_DB=inventory
```

```
volumes: { caddy_data: {}, pg_data: {} }
```

11.4 Caddyfile

```
inventory.mypc.tail1234.ts.net {  
  tls /certs/cert.pem /certs/key.pem  
  
  handle /api/* {  
    reverse_proxy nestjs:4000  
  }  
  
  handle {  
    reverse_proxy nextjs:3000  
  }  
}
```

11.5 Folder Structure on Server

```
/home/user/  
├── inventory/  
│   ├── docker-compose.yml  
│   ├── Caddyfile  
│   ├── certs/                ← tailscale cert output  
│   │   ├── cert.pem  
│   │   └── key.pem  
│   ├── frontend/            ← Next.js source  
│   │   └── Dockerfile  
│   ├── backend/             ← NestJS source  
│   │   └── Dockerfile  
│   ├── data/  
│   │   └── uploads/         ← item images  
└── (future-app-2)/  
    └── docker-compose.yml
```

12. Raspberry Pi Migration (Future)

When you're ready to move from the laptop to a Raspberry Pi, the migration is straightforward because everything is containerised. Nothing in the app code changes — only where it runs.

12.1 Recommended Hardware

Item	Recommendation	Approx. Cost	Notes
Pi board	Raspberry Pi 5 (8GB)	~80€	8GB RAM comfortably runs 6-8 apps simultaneously
Power supply	Official Pi 5 27W USB-C	~12€	Do not cheap out — underpowered PSU causes crashes
Storage	USB 3.0 SSD (256GB+)	~25€	Never run Postgres on an SD card — it will die
Case	Any with active cooling	~10€	Pi 5 runs warm under load; a fan helps
SD card	Any 16GB+	~8€	Just for initial OS install; boot target moves to SSD

12.2 Migration Steps

- Install Raspberry Pi OS Lite (64-bit) on the SD card using Raspberry Pi Imager.
- Boot, run `sudo raspi-config` to enable SSH, then move the boot target to the USB SSD.
- Install Docker and Docker Compose on the Pi (official Docker install script works on ARM64).
- Install Tailscale on the Pi — it joins your existing Tailscale network automatically.
- Run `tailscale cert` to generate a new cert for the Pi's Tailscale hostname.
- Copy your `docker-compose.yml`, `Caddyfile`, and update the hostname in the `Caddyfile`.
- Run `pg_dump` on the laptop to export the database, restore it on the Pi with `pg_restore`.
- `docker compose up -d` — everything starts. Update the Tailscale hostname on your phones.
- The laptop setup can be shut down. Total migration time: approximately 2 hours.

12.3 Running Multiple Apps on the Pi

Each additional app gets its own folder and `docker-compose.yml`. Caddy is shared across all apps via a single `Caddyfile`, adding one block per app:

```
# Add to Caddyfile for each new app:
recipes.mypi.tail1234.ts.net {
  tls /certs/cert.pem /certs/key.pem
  reverse_proxy recipes-app:3001
```

```
}  
  
photos.mypi.tail1234.ts.net {  
    tls /certs/cert.pem /certs/key.pem  
    reverse_proxy immich-server:2283  
}
```

Popular self-hosted apps that run well alongside your inventory system on a Pi 5 8GB:

- Immich — self-hosted Google Photos replacement
- Vaultwarden — Bitwarden-compatible password manager
- Home Assistant — smart home automation hub
- Paperless-ngx — document scanning and OCR storage
- Uptime Kuma — lightweight monitoring for all your services

— *End of Document* —